

Sobre o Git

Aprenda sobre o sistema de controle de versões Git e como ele funciona com o GitHub.

Sobre controle de versão e Git

Um sistema de controle de versões, ou VCS, rastreia o histórico de alterações à medida que pessoas e equipes colaboram em projetos. Conforme os desenvolvedores fazem alterações no projeto, qualquer versão anterior pode ser recuperada a qualquer momento.

Os desenvolvedores podem consultar o histórico do projeto para descobrir:

- Quais mudanças foram feitas?
- Quem fez as alterações?
- Quando foram feitas as alterações?
- Por que as mudanças eram necessárias?

Os Sistemas de Controle de Versões (VCSs) oferecem a cada colaborador uma visão unificada e consistente do projeto, revelando o trabalho já em andamento. Visualizar um histórico transparente das alterações, quem as fez e como elas contribuem para o desenvolvimento do projeto ajuda os membros da equipe a manterem-se alinhados, mesmo trabalhando de forma independente.

Em um sistema de controle de versão distribuído, cada desenvolvedor possui uma cópia completa do projeto e de seu histórico. Diferentemente dos sistemas de controle de versão centralizados, outrora populares, os DVCSs não necessitam de uma conexão constante com um repositório central. O Git é o sistema de controle de versão distribuído mais popular. Ele é comumente utilizado tanto para o desenvolvimento de software de código aberto quanto para o desenvolvimento de software comercial, oferecendo benefícios significativos para indivíduos, equipes e empresas.

- O Git permite que os desenvolvedores vejam toda a linha do tempo de suas alterações, decisões e progresso de qualquer projeto em um só lugar. A partir do momento em que acessam o histórico de um projeto, o desenvolvedor tem todo o contexto necessário para entendê-lo e começar a contribuir.
- Os desenvolvedores trabalham em todos os fusos horários. Com um DVCS como o Git, a colaboração pode acontecer a qualquer momento, mantendo a integridade do código-fonte. Usando branches, os desenvolvedores podem propor alterações no código de produção com segurança.

- Empresas que utilizam Git podem eliminar as barreiras de comunicação entre equipes e mantê-las focadas em realizar o seu melhor trabalho. Além disso, o Git possibilita o alinhamento de especialistas de toda a empresa para colaborarem em projetos importantes.

Sobre repositórios

Um repositório, ou projeto Git, engloba toda a coleção de arquivos e pastas associados a um projeto, juntamente com o histórico de revisões de cada arquivo. O histórico de arquivos aparece como instantâneos no tempo, chamados commits. Os commits podem ser organizados em múltiplas linhas de desenvolvimento, chamadas branches. Como o Git é um DVCS (Sistema de Controle de Versões Distribuído), os repositórios são unidades autocontidas e qualquer pessoa que possua uma cópia do repositório pode acessar todo o código-fonte e seu histórico. Usando a linha de comando ou outras interfaces fáceis de usar, um repositório Git também permite: interação com o histórico, clonagem do repositório, criação de branches, commits, merges, comparação de alterações entre versões do código e muito mais.

Por meio de plataformas como o GitHub, o Git também oferece mais oportunidades para transparência e colaboração em projetos. Repositórios públicos ajudam as equipes a trabalharem juntas para construir o melhor produto final possível.

Como funciona o GitHub

O GitHub hospeda repositórios Git e fornece aos desenvolvedores ferramentas para entregar código melhor por meio de recursos de linha de comando, issues (discussões em tópicos), pull requests, revisão de código ou o uso de uma coleção de aplicativos gratuitos e pagos no GitHub Marketplace. Com camadas de colaboração como o GitHub Flow, uma comunidade de 100 milhões de desenvolvedores e um ecossistema com centenas de integrações, o GitHub muda a forma como o software é construído.

O GitHub integra a colaboração diretamente ao processo de desenvolvimento. O trabalho é organizado em repositórios onde os desenvolvedores podem definir requisitos ou direcionamento e estabelecer expectativas para os membros da equipe. Em seguida, usando o fluxo do GitHub, os desenvolvedores simplesmente criam uma branch para trabalhar nas atualizações, confirmam as alterações para salvá-las, abrem um pull request para propor e discutir mudanças e mesclam os pull requests quando todos estiverem alinhados. Para mais informações, consulte [o fluxo do GitHub](#).

Para informações sobre planos e custos do GitHub, consulte [Preços do GitHub](#). Para informações sobre como o GitHub Enterprise se compara a outras opções, consulte [Comparando o GitHub com outras soluções de DevOps](#).

GitHub e a linha de comando

Comandos básicos do Git

Para usar o Git, os desenvolvedores utilizam comandos específicos para copiar, criar, alterar e combinar código. Esses comandos podem ser executados diretamente na linha de comando ou por meio de um aplicativo como o GitHub Desktop. Aqui estão alguns comandos comuns para usar o Git:

- `git init` Inicializa um novo repositório Git e começa a rastrear um diretório existente. Adiciona uma subpasta oculta dentro do diretório existente que contém a estrutura de dados interna necessária para o controle de versão.
- `git clone` Cria uma cópia local de um projeto que já existe remotamente. O clone inclui todos os arquivos, histórico e branches do projeto.
- `git add` O Git rastreia as alterações no código-fonte de um desenvolvedor, mas é necessário preparar e criar um snapshot das alterações para incluí-las no histórico do projeto. Este comando realiza a preparação, a primeira parte desse processo de duas etapas. Quaisquer alterações preparadas farão parte do próximo snapshot e do histórico do projeto. Preparar e confirmar as alterações separadamente dá aos desenvolvedores controle total sobre o histórico do projeto sem alterar a forma como codificam e trabalham.
- `git commit` Salva o snapshot no histórico do projeto e conclui o processo de rastreamento de alterações. Em resumo, um commit funciona como tirar uma foto. Tudo o que foi preparado com ``git commit`` `git add` se tornará parte do snapshot com ``git commit`` `git commit`.
- `git status` Mostra o status das alterações como não rastreadas, modificadas ou em fase de preparação.
- `git branch` Mostra as filiais em que estão sendo feitas obras locais.
- `git merge` Mescla linhas de desenvolvimento. Este comando é normalmente usado para combinar alterações feitas em duas ramificações distintas. Por exemplo, um desenvolvedor usaria o comando ``merge`` quando desejasse combinar as alterações de uma ramificação de recurso na ramificação principal para implantação.
- `git pull` Atualiza a linha de desenvolvimento local com as atualizações da sua contraparte remota. Os desenvolvedores usam este comando se um colega de equipe fez commits em um branch remoto e eles gostariam de refletir essas alterações em seu ambiente local.
- `git push` Atualiza o repositório remoto com quaisquer commits feitos localmente em um branch.

